

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА – Российский технологический университет"

РТУ МИРЭА

Институт кибербезопасности и цефровых технологий (ИКБ)
Кафедра КБ-9 “Предметно-ориентированные информационные системы”

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
по дисциплине
«Интеллектуальные системы и технологии»

Выполнили студенты группы БСМО-11-26 — Паскевичян А. А., Васильев А. В.

Принял кандидат технических наук, доцент кафедры КБ-9 — Судаков В. А.

Москва 2026

1. Описание датасета

В качестве датасета был выбран готовый открытый датасет wine-quality-white-and-red. Исходный набор содержит 6497 записей и 13 столбцов. Целевой признак type принимает два значения: white — белое вино и red — красное вино. В исходных данных также присутствует признак quality, однако перед обучением он был удалён и в классификации не использовался. Распределение классов: 4898 объектов белого вина (75,39 %) и 1599 объектов красного вина (24,61 %).

Описание данных датасета:

1. Fixed acidity — фиксированная кислотность. Большинство кислот, содержащихся в вине, являются фиксированными или нелетучими, то есть они не испаряются легко.

2. Volatile acidity — летучая кислотность. Количество уксусной кислоты в вине. При слишком высокой концентрации она может придавать вину неприятный уксусный вкус.

3. Citric acid — лимонная кислота. Содержится в небольших количествах и может придавать вину свежесть и улучшать его вкусовые качества.

4. Residual sugar — остаточный сахар. Количество сахара, оставшегося после завершения процесса брожения.

5. Chlorides — хлориды. Содержание солей в вине.

6. Free sulfur dioxide — свободный диоксид серы (SO_2). Количество свободной формы диоксида серы. Используется для защиты вина от окисления и развития нежелательных микроорганизмов.

7. Total sulfur dioxide — общий диоксид серы (SO_2). Общее количество свободной и связанной форм диоксида серы в вине.

8. Density — плотность. Плотность вина, которая зависит в том числе от содержания алкоголя и сахара.

9. pH — показатель кислотности. Характеризует кислотность или щёлочность вина по шкале от 0 до 14. Для большинства вин значение pH находится примерно в диапазоне 3–4.

10. Sulphates — сульфаты. Добавки, которые могут увеличивать содержание соединений серы и способствуют антимикробной защите вина.

11. Alcohol — содержание алкоголя. Процентное содержание алкоголя в вине.

2. Подготовка параметров и целевого признака

Целевой признак был преобразован в числовой вид: red = 0, white = 1. Далее выборка была случайным образом перемешана с фиксированным seed = 42 и разделена на обучающую и тестовую части в соотношении 80/20. В обучающую выборку вошло 5198 объектов, в тестовую — 1299 объектов (рисунок 1).

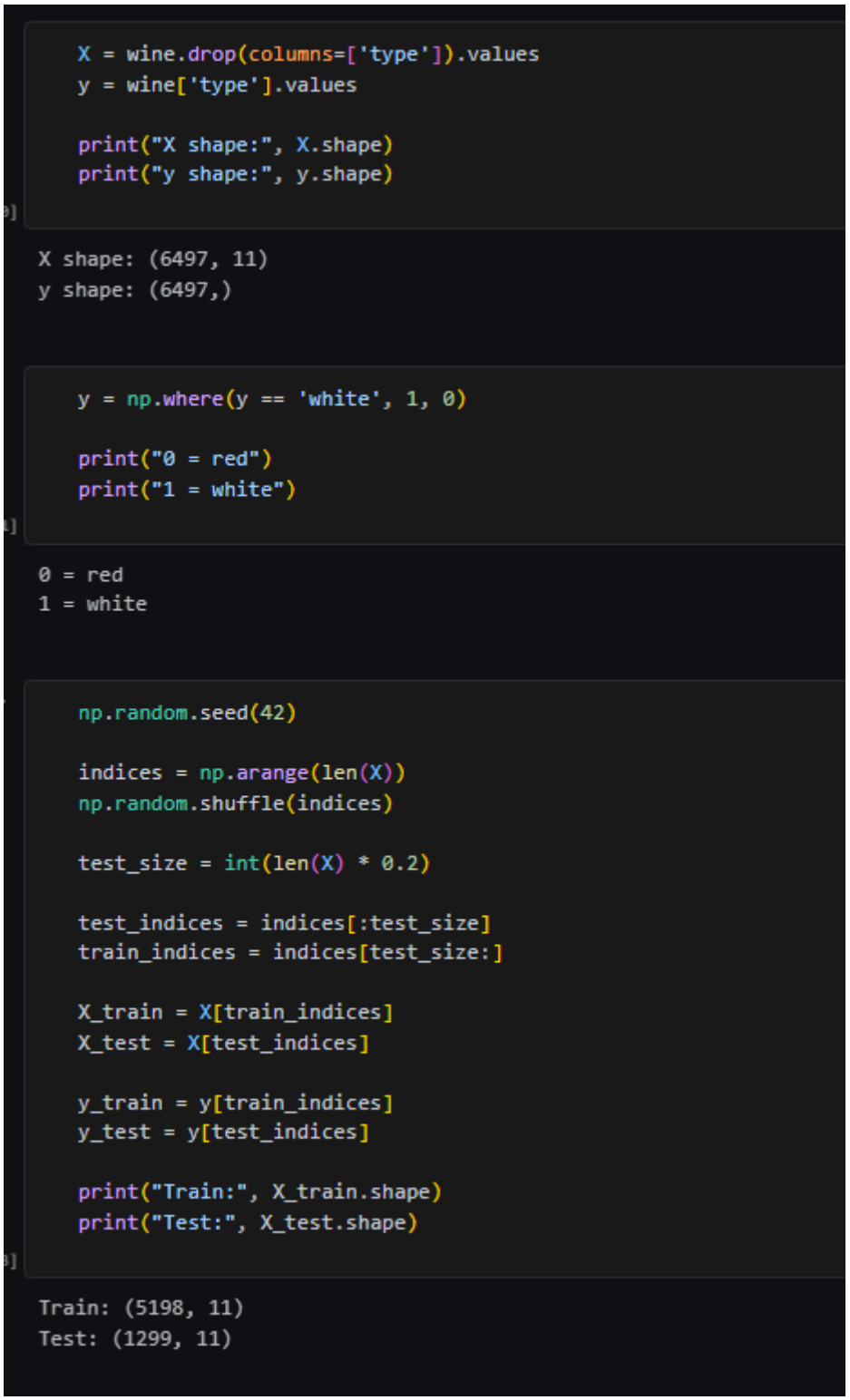


Рисунок 1 – Подготовка параметров

Перед вычислением расстояний выполнена Z-нормализация признаков (рисунок 2). Для каждого признака из значения вычиталось среднее по обучающей выборке, после чего результат делился на стандартное отклонение. Нормализация нужна для того, чтобы признаки с большими

числовыми значениями не оказывали непропорционально сильного влияния на расстояние между объектами.

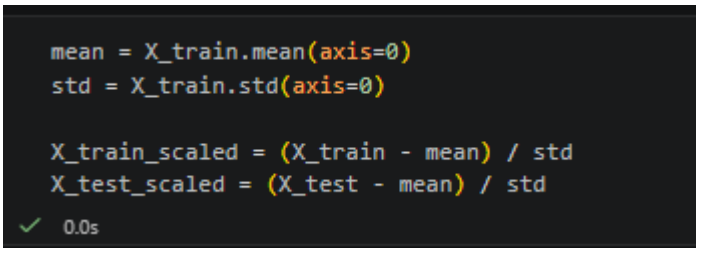


Рисунок 2 – Нормализация признаков

3. Реализация метода k-ближайших соседей

```python
def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))
```

✓ 0.0s

Weighted k-NN

$w_i = \frac{1}{d_i + \varepsilon}$

где:

d_i — расстояние до соседа; w_i — его вес; ε — маленькое число, чтобы избежать деления на ноль.

```python
def knn_predict_one(X_train, y_train, x, k):
    distances = []

    # Считаем расстояния до всех объектов train
    for i in range(len(X_train)):
        distance = euclidean_distance(X_train[i], x)
        distances.append((distance, y_train[i]))

    # Сортируем по расстоянию
    distances.sort(key=lambda item: item[0])

    # Берём k ближайших соседей
    neighbors = distances[:k]

    # Суммарный вес для каждого класса
    class_weights = {
        0: 0.0,  # red
        1: 0.0   # white
    }

    epsilon = 1e-10

    for distance, label in neighbors:
        weight = 1 / (distance + epsilon)
        class_weights[label] += weight

    # Выбираем класс с максимальным суммарным весом
    prediction = max(
        class_weights,
        key=class_weights.get
    )

    return prediction
```

✓ 0.0s

Рисунок 3 – Реализация функции классификации и функции Евклидова расстояния

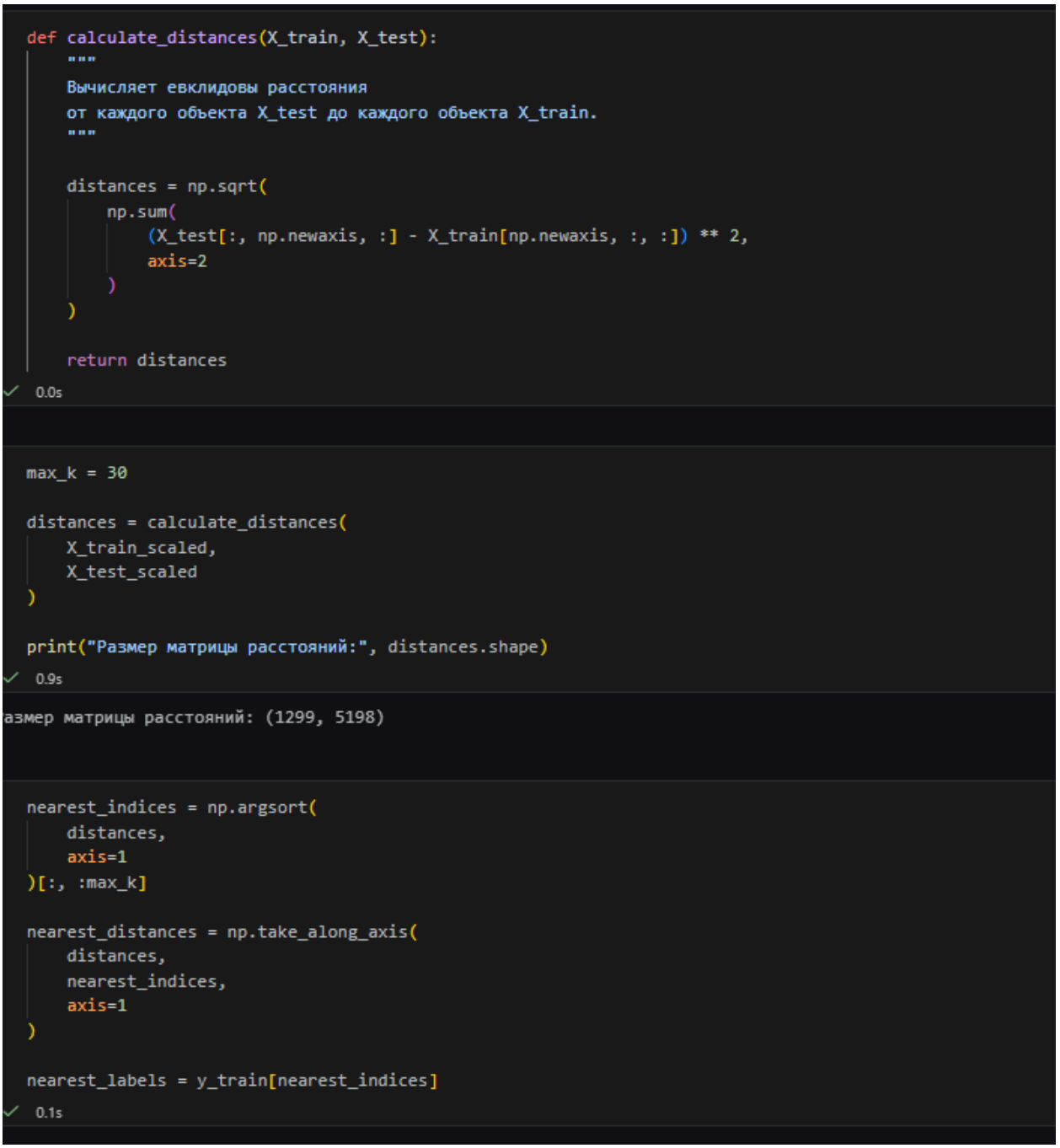


Рисунок 4 – Подсчет расстояний между объектами тестовой и обучающей выборок

```python
def weighted_knn_predict(nearest_distances, nearest_labels, k):

    distances = nearest_distances[:, :k]
    labels = nearest_labels[:, :k]

    # Веса соседей
    weights = np.zeros_like(distances, dtype=float)

    # Обычный случай: weight = 1 / distance
    non_zero = distances > 0
    weights[non_zero] = 1 / distances[non_zero]

    # Если расстояние равно 0,
    # такой сосед должен иметь максимальный вес
    weights[~non_zero] = 1e12

    # Сумма весов для red (0)
    red_weight = np.sum(
        weights * (labels == 0),
        axis=1
    )

    # Сумма весов для white (1)
    white_weight = np.sum(
        weights * (labels == 1),
        axis=1
    )

    # Выбираем класс с большим суммарным весом
    predictions = (white_weight > red_weight).astype(int)

    return predictions
```

Рисунок 5 – Взвешенная классификация k-nn

4. Проверка работы алгоритма

Для демонстрации работы алгоритма в программе рассматривается отдельная тестовая точка и её пять ближайших соседей. В выбранном примере все пять соседей относятся к классу white (рисунок 6).

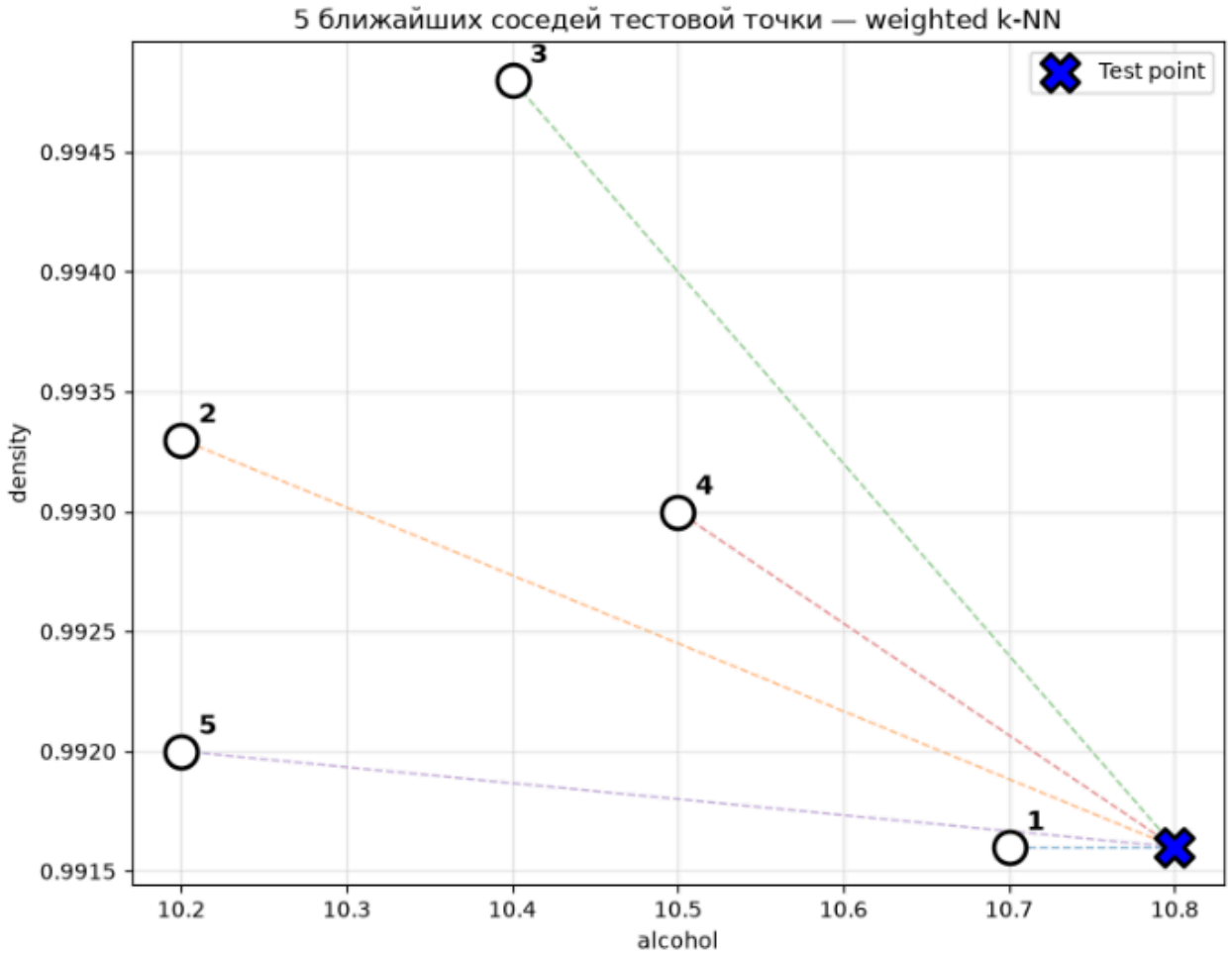


Рисунок 6 – Проверка работы алгоритма

5. Подбор оптимального значения k

Для выбора значения k был выполнен перебор значений от 1 до 30 (рисунок 7). Для каждого k программа формировала предсказания для всей тестовой выборки и вычисляла долю правильных ответов (Accuracy). Наилучший результат получен при k = 3: Accuracy = 0,9962, то есть 99,62 % (рисунок 8).

```python
k_values = range(1, 31)

accuracies = []

for k in k_values:

    y_pred = weighted_knn_predict(
        nearest_distances,
        nearest_labels,
        k
    )

    accuracy = accuracy_score_manual(
        y_test,
        y_pred
    )

    accuracies.append(accuracy)

    print(f"k = {k:2d} | Accuracy = {accuracy:.4f}")
```

```
k =  1 | Accuracy = 0.9938
k =  2 | Accuracy = 0.9938
k =  3 | Accuracy = 0.9962
k =  4 | Accuracy = 0.9954
k =  5 | Accuracy = 0.9938
k =  6 | Accuracy = 0.9938
k =  7 | Accuracy = 0.9938
k =  8 | Accuracy = 0.9938
k =  9 | Accuracy = 0.9938
k = 10 | Accuracy = 0.9946
k = 11 | Accuracy = 0.9938
k = 12 | Accuracy = 0.9923
k = 13 | Accuracy = 0.9923
k = 14 | Accuracy = 0.9923
k = 15 | Accuracy = 0.9923
k = 16 | Accuracy = 0.9915
k = 17 | Accuracy = 0.9923
k = 18 | Accuracy = 0.9923
k = 19 | Accuracy = 0.9923
k = 20 | Accuracy = 0.9908
k = 21 | Accuracy = 0.9908
k = 22 | Accuracy = 0.9915
k = 23 | Accuracy = 0.9915
k = 24 | Accuracy = 0.9915
k = 25 | Accuracy = 0.9923
...
k = 27 | Accuracy = 0.9915
k = 28 | Accuracy = 0.9915
k = 29 | Accuracy = 0.9915
k = 30 | Accuracy = 0.9915
```

Рисунок 7 – Подбор лучшего k

```
best_index = np.argmax(accuracies)

best_k = list(k_values)[best_index]
best_accuracy = accuracies[best_index]

print()
print(f"Лучший k: {best_k}")
print(f"Лучшая Accuracy: {best_accuracy:.4f}")
print(f"Лучшая Accuracy: {best_accuracy * 100:.2f}%")
```

```
Лучший k: 3
Лучшая Accuracy: 0.9962
Лучшая Accuracy: 99.62%
```

Рисунок 8 – Лучшее k

На рисунке 9 отражена зависимость Accurancy от значения k.

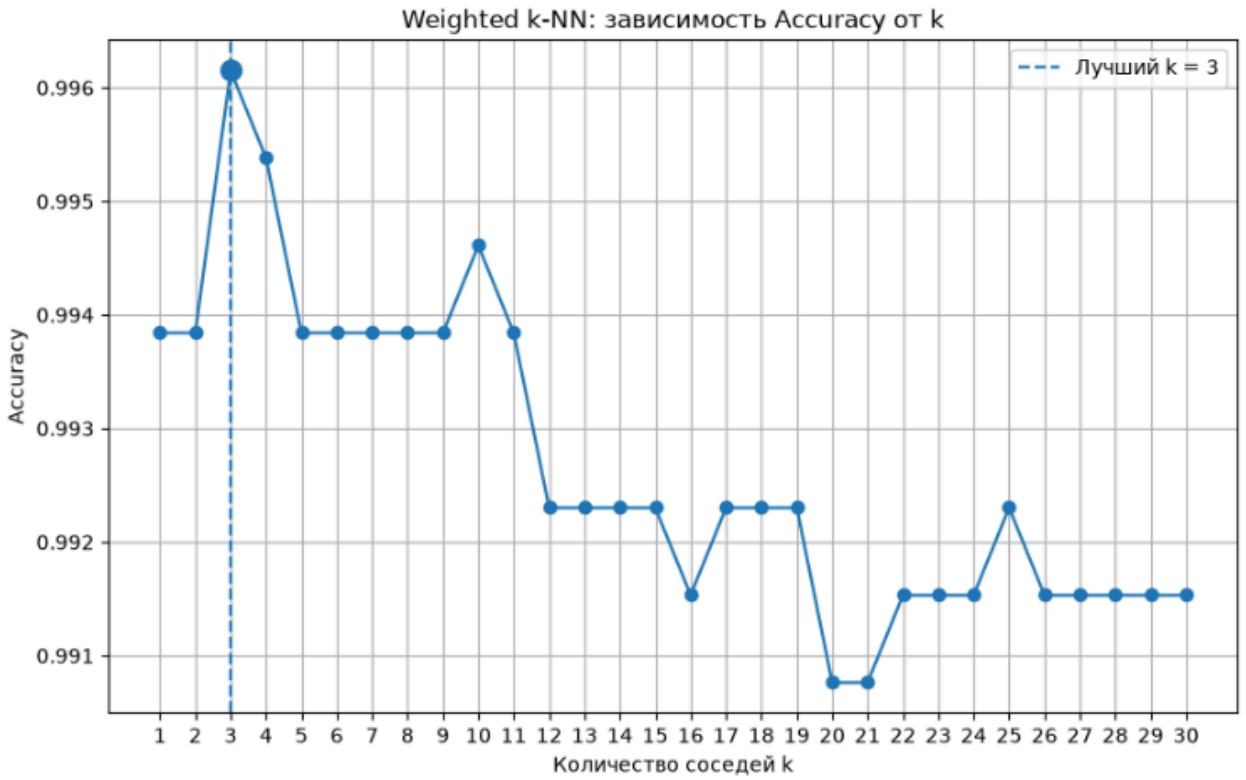


Рисунок 9 - зависимость Accurancy от значения k